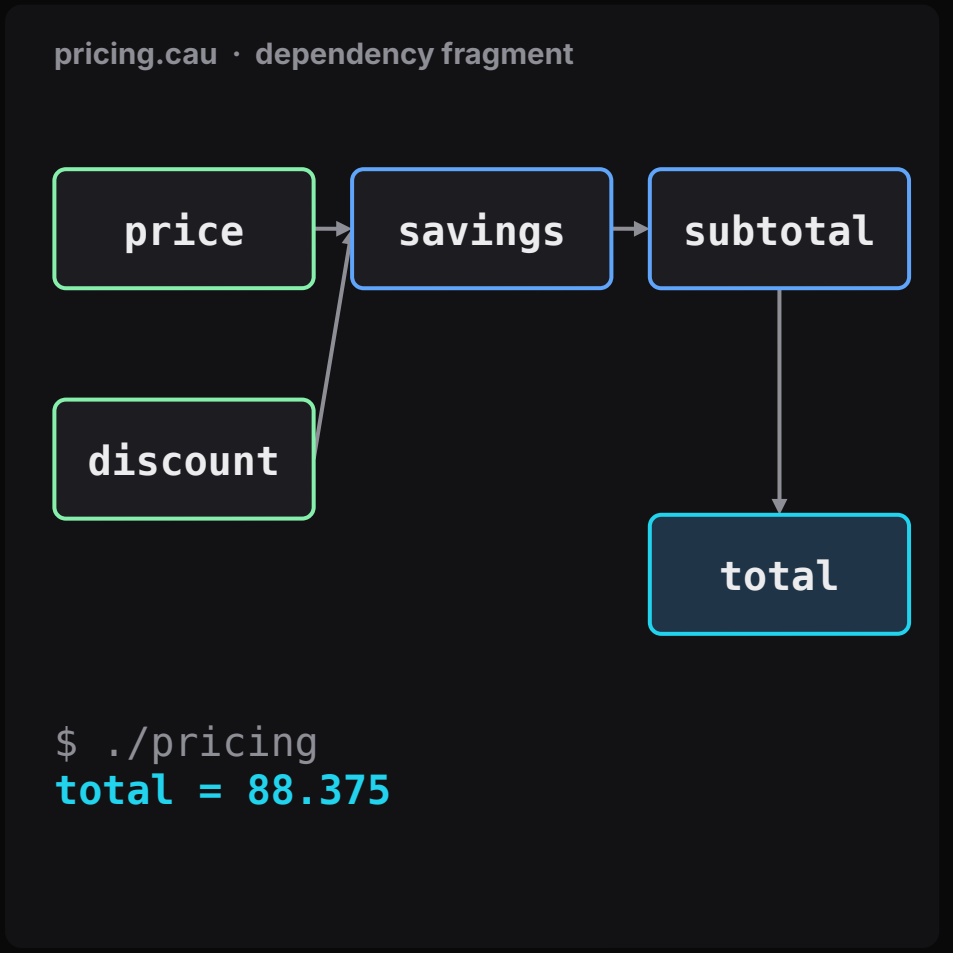
CAUSA

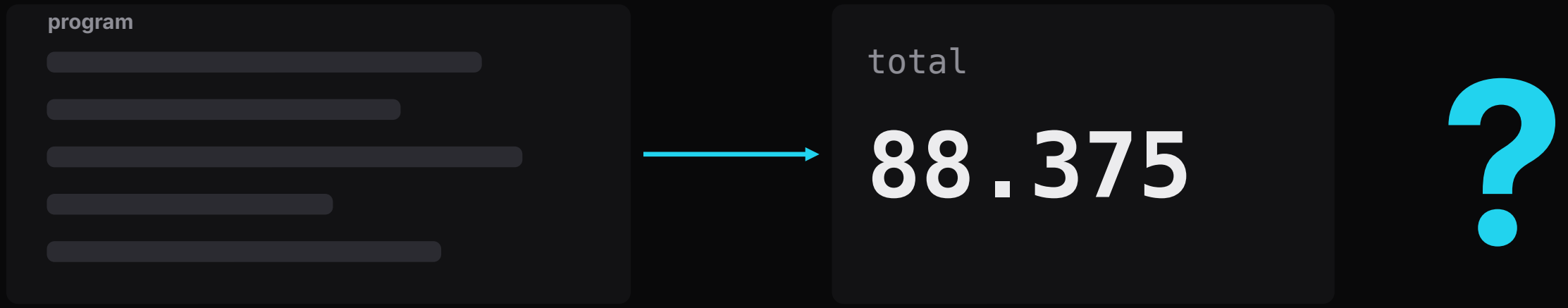
A Compiler for Explainable and Counterfactual Programming

Compiler Design Project · Final Review

[Student name] · [Registration number]
[School / University]
Guide: [Faculty guide name]



A result appears. Its reasoning does not



1

Why did this result occur?

2

What depends on this input?

3

What changes if this value changes?

Objectives: dependencies as a language feature

Motivation Understanding a computed value usually means tracing code by hand.

1

Source-level dependency awareness

Each binding records the names its expression uses.

✓ Implemented

`semantic.py · ir.py`

2

Explainable calculations

explain prints a derivation tree with values and source expressions.

✓ Implemented

`runtime: causa_explain`

3

Impact analysis

impact lists direct and all dependents, known statically.

✓ Implemented

`runtime: causa_impact`

4

Isolated what-ifs

whatif recomputes in a separate environment; real values stay.

✓ Implemented

`runtime: causa_whatif`

5

Native executables

A Python compiler emits C99; a system C compiler builds the program.

✓ Implemented

`codegen.py · runtime`

Manual tracing vs built-in operations

	Conventional workflow	CAUSA
Understand a derivation	Read the code; add prints or step through a debugger	`explain total` a derivation tree with values and source expressions
Find downstream effects	Search for uses by hand or with editor tooling	`impact price` direct and all dependent bindings
Try a changed input	Edit the value, rerun, compare outputs	`whatif price = 160` before → after; original values untouched
Keep dependencies through optimization	Depends on the tool; not a language guarantee	`verified optimizer` binding-level graph re-checked after optimization

Conventional tools can do all of this. CAUSA makes it explicit and built into the language and runtime.

CAUSA: a language and a compiler

show	print a binding's value
explain	derivation tree for a binding
impact	direct and all dependents
whatif	recompute under hypothetical inputs
assert	check a condition at run time

example.cau

```
input price = 100
input discount = 0.25

let savings = price * discount
let total = price - savings

show total                    # total = 75.0
explain total
whatif price = 160 show total
impact price
```

A custom language and compiler (Python → C99 → native executable): not a Python library, not a graph viewer.

Related work and foundations

Dependency analysis

[1] Weiser 1984
program slicing
Foundational

[2] Ferrante et al. 1987
program dependence graph
Foundational

Which statements affect a value

Provenance

[3] Cheney et al. 2009
why, how, where provenance
Directly related

[4] Cheney et al. 2011
provenance as dependency analysis
Directly related

Dependencies explain results

Explanations and counterfactuals

[5] Ko & Myers 2004
Whyline: “why” questions
Directly related

[6] Karimi et al. 2022
counterfactual explanations survey
Recent · related

Asking why, and what if

Compilers and testing

[7] Lattner & Adve 2004
LLVM IR

[8] Lattner et al. 2021
MLIR

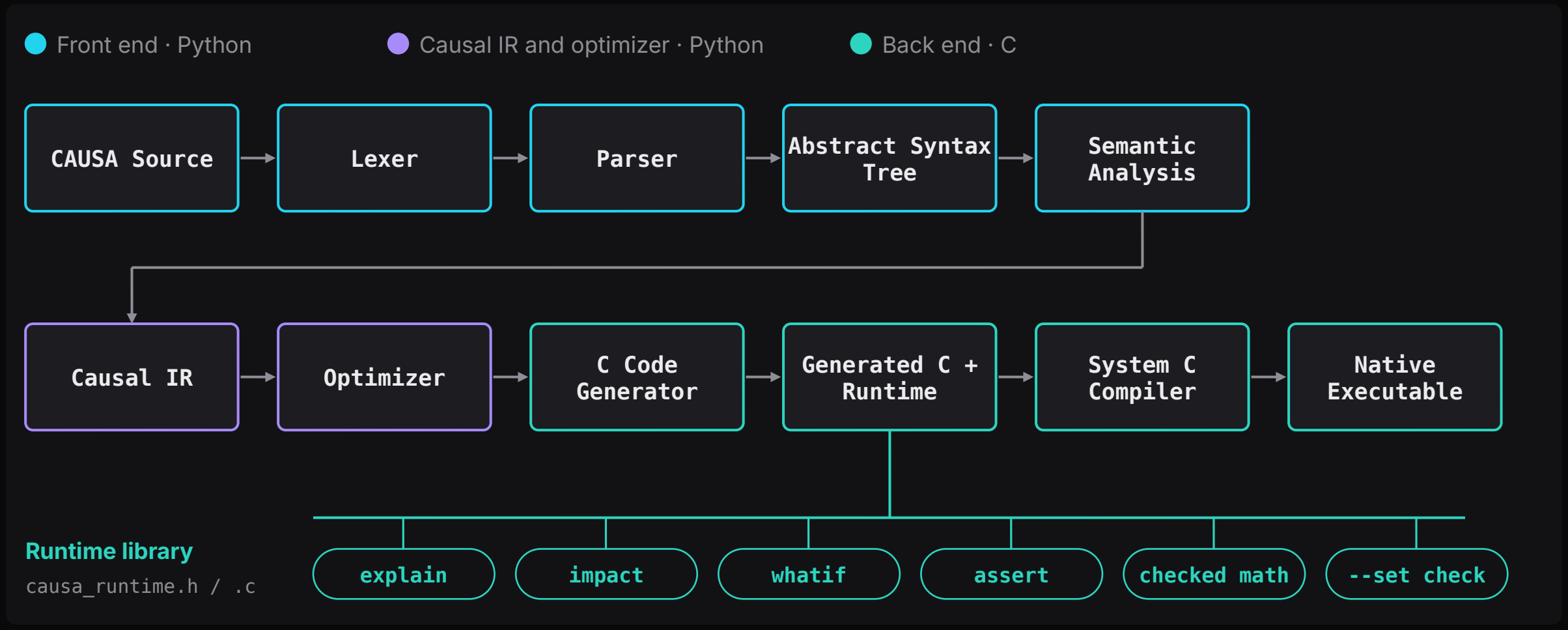
[9] Chen et al. 2020
compiler testing survey

[10] Ma 2023 (preprint)
compiler fuzzing survey

IR design and validation

CAUSA builds on established ideas; its contribution is integrating them into one language, compiler and runtime workflow.

System architecture



Dependencies are computed at compile time and embedded as metadata; the executable records values.

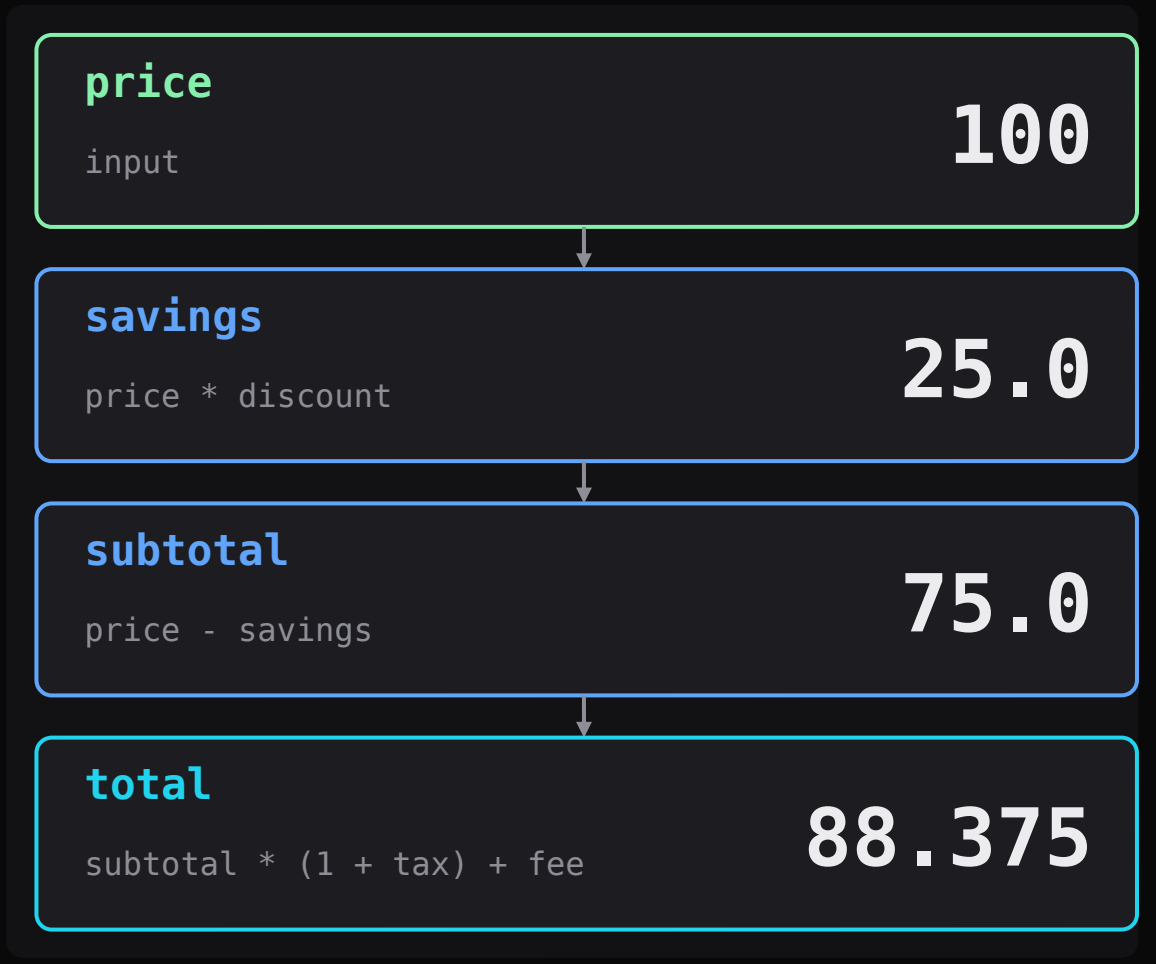
Language example: a pricing model

pricing.cau (excerpt)

```
input price = 100
input discount = 0.25
input tax = 0.125
input fee = 4

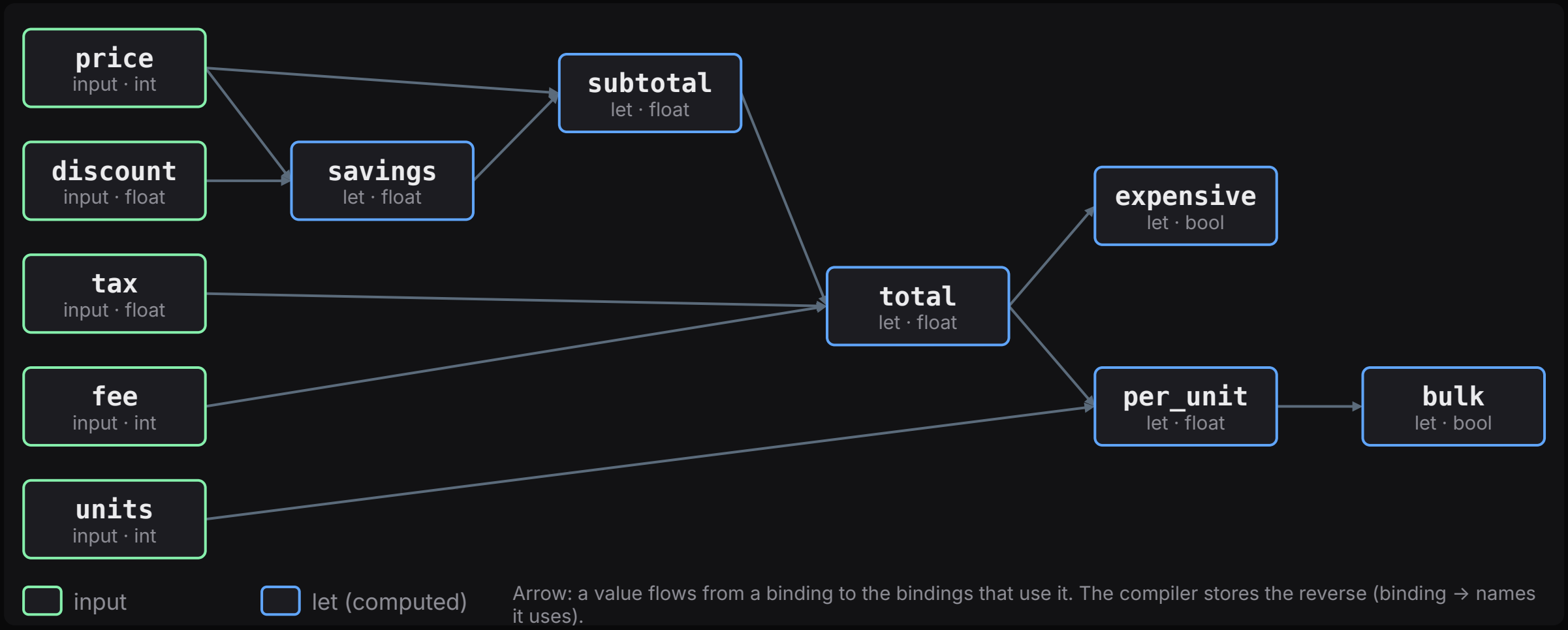
let savings = price * discount
let subtotal = price - savings
let total = subtotal * (1 + tax) + fee

show total
```



100 → 25.0 savings → 75.0 subtotal → 88.375 total

The dependency graph: 11 bindings, 11 edges



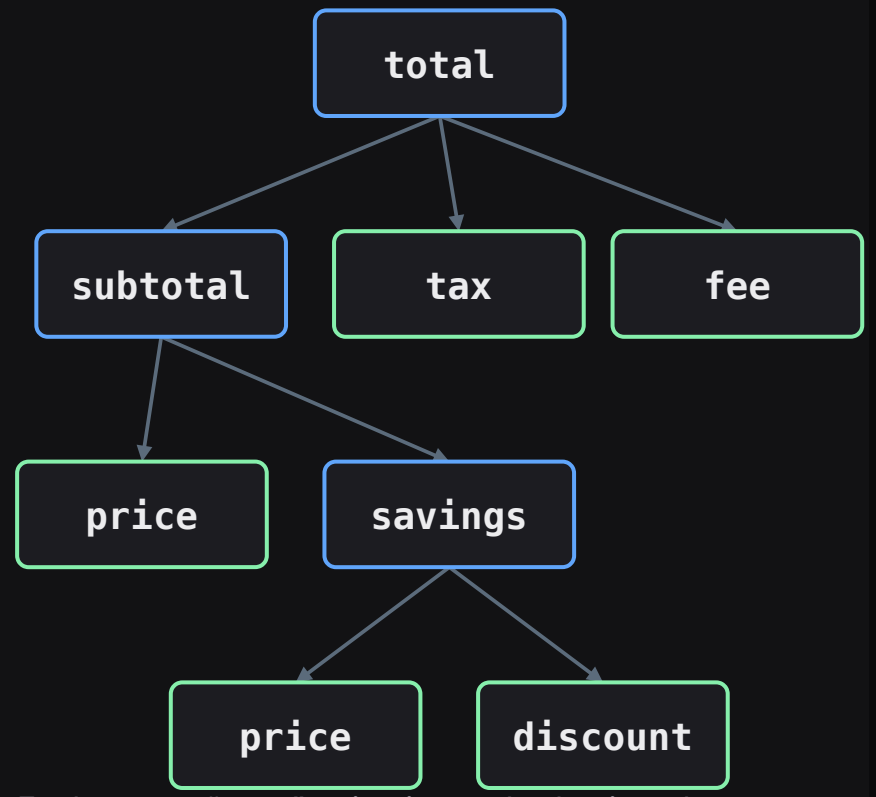
Dependencies are derived from source expressions, not from whether the current numerical output changes.

explain: why did this value occur?

$ ./build/pricing (output of explain total)

```
explain total
  total = 88.375  [let: subtotal * (1 + tax) + fee]
    subtotal = 75.0  [let: price - savings]
      price = 100  [input]
      savings = 25.0  [let: price * discount]
        price = 100  [input, see above]
        discount = 0.25  [input]
    tax = 0.125  [input]
    fee = 4  [input]
  direct dependencies: tax, fee, subtotal
  transitive dependencies: price, discount, tax,
      fee, savings, subtotal
  inputs: price, discount, tax, fee
  intermediates: savings, subtotal
```

depends on



Each arrow: “uses”. price is reached twice; the runtime prints it once and marks it “see above”.

impact: what else changes?

output of `impact discount`

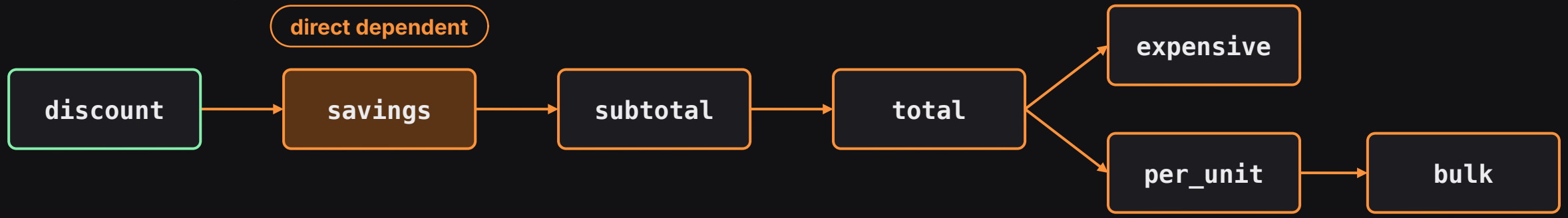
```
impact discount
  direct dependents: savings
  all dependents: savings, subtotal, total,
      expensive, per_unit, bulk
```

Two answers

direct: bindings whose expression mentions discount

all: those plus everything downstream of them

all affected: savings, subtotal, total, expensive, per_unit, bulk



Static: read from compile-time dependency tables. No values are recomputed.

whatif: counterfactual execution

binding	baseline		whatif price = 150
price	100	→	**150**
savings	25.0	→	**37.5**
subtotal	75.0	→	**112.5**
total	88.375	→	**130.5625**
expensive	true	→	**true**
per_unit	22.09375	→	**32.640625**
bulk	true	→	**false**

Isolated

A separate environment is recomputed; the real execution is never written. After five whatif statements, show total still prints 88.375.

launch-time inputs

```
$ ./build/pricing \
    --set price=150 \
    --set discount=0.3
total = 122.125
```

Source: values from the real runtime, shown in CAUSA Studio's What-if panel; launch output from docs/demonstration.md.

changed input → affected calculations recomputed → resulting output

Causal IR: optimization without losing dependencies

Causal IR (price * (1 - discount))

```
n0 = input     int    price
n1 = input     float  discount
n2 = const     int    1
n3 = itof      n2
n4 = sub       n3, n1
n5 = itof      n0
n6 = mul       n5, n4
n7 = bind      n6     total

total → price, discount
```

Optimizer

Allowed

- fold constants that cannot fail
- merge duplicates in one binding
- remove dead constants

Never

- drop an input or binding
- remove a reference to a binding
- fold 1 / 0 or an overflow

pricing.cau: 30 → 28 nodes

Design invariant

```
let y = x * 0
let z = x - x
```

A numeric simplifier would turn both into 0 and erase the edge to x.

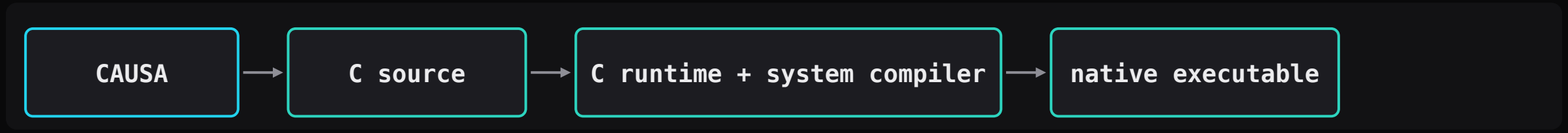
CAUSA keeps it:

```
y: let → x
z: let → x
optimizer: no changes
```

Illustrates the invariant; checked by running the compiler.

After optimizing, the binding-level graph is re-derived and must equal the original.

Code generation and the runtime



generated C (excerpt of pricing.c)

```
case 6: /* n6 op mul @10:21 owner savings */
  return causa_check(
      causa_mul_f64(env[5].f, env[1].f, &env[6].f),
      10, 21, err);
case 7: /* n7 bind savings: price * discount */
  env[7] = env[6];
  return CAUSA_OK;

causa_configure(argc, argv);   /* --set NAME=VALUE */
causa_explain(env, CAUSA_BINDING_total);
```

Checked arithmetic

overflow detected before computing; R101–R103 report the operator's line and column

Exact floats

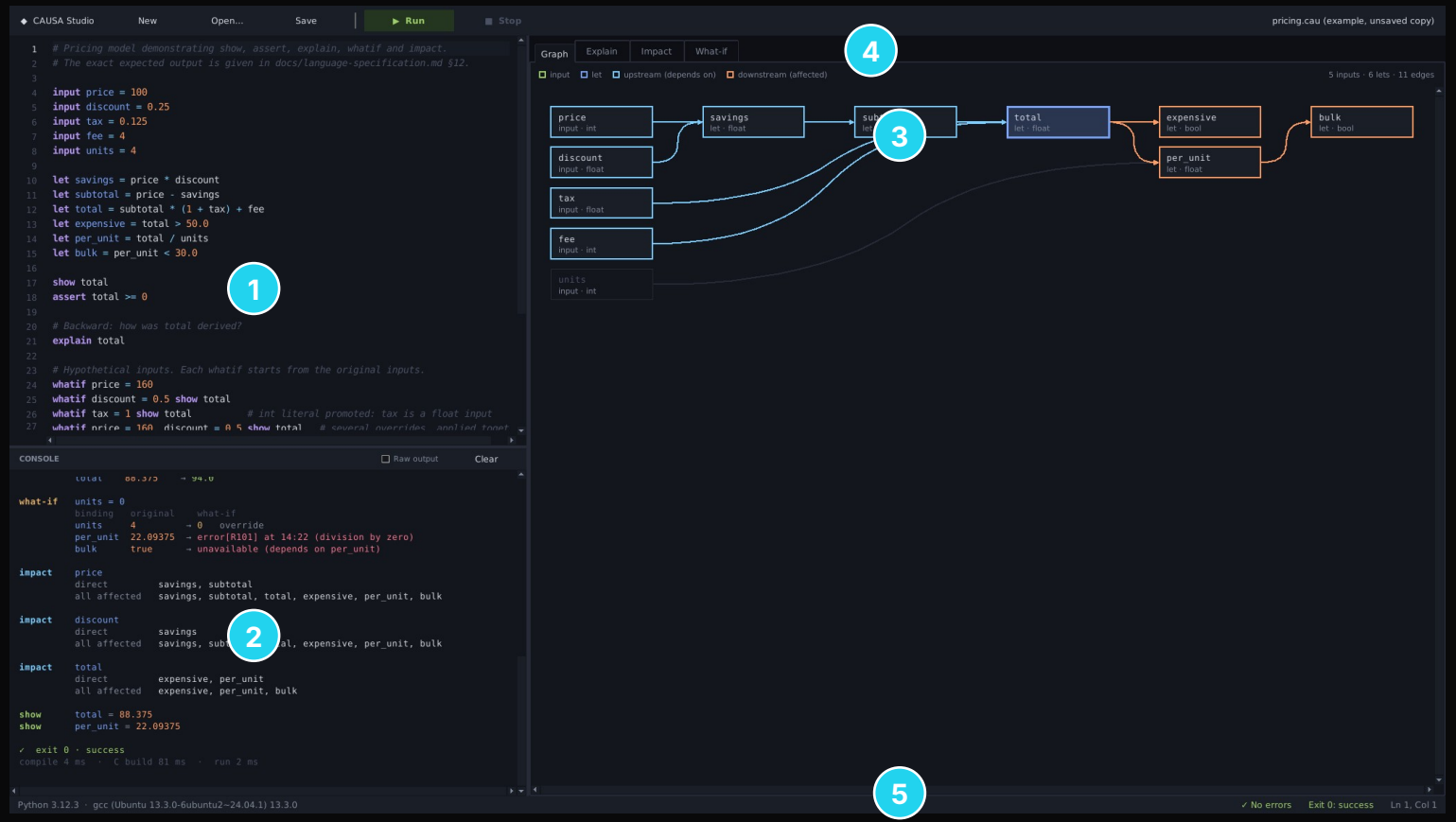
literals emitted as hex floating constants; no decimal rounding in the C compiler

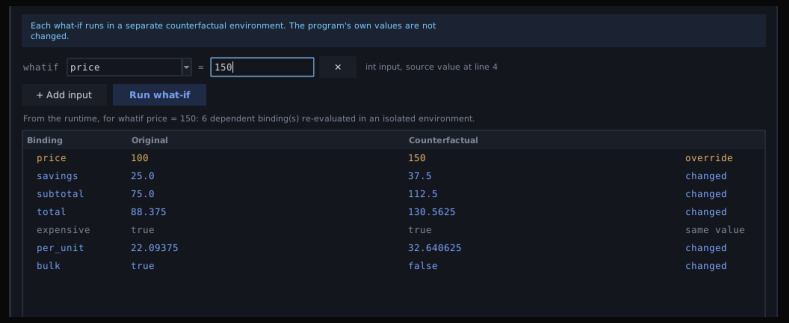
Small runtime

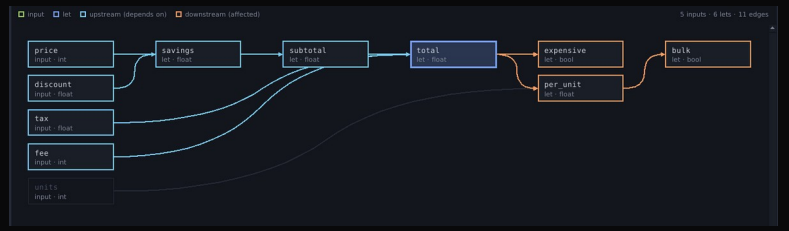
explain, impact, whatif, assert from metadata tables; standard C library only

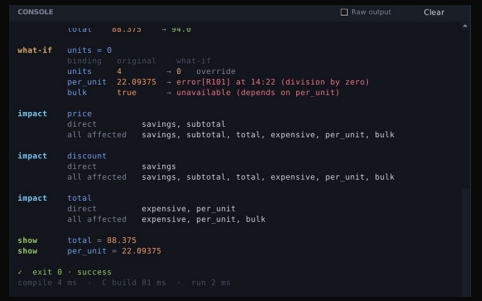
```
Build:  gcc -std=c99 -I causa/runtime pricing.c causa/runtime/causa_runtime.c -o pricing -lm
```

CAUSA Studio: the desktop application









1 Source editor 2 Integrated console 3 Dependency graph 4 Analysis tabs 5 Build / runtime status

Real application screenshots, captured by presentation/capture_studio.py (Linux, Xvfb, Python 3.12). Pricing example: 5 inputs · 6 lets · 11 edges.

Testing and validation

205

automated tests passed

0 skipped · recorded full run

Windows 11 · Python 3.11.9 · MSYS2 UCRT64 GCC 16.1.0

Current main has 257 tests: these 205 plus 52 Studio tests.

Strict C99

Generated C compiles warning-free with -Wall -Wextra -Wpedantic -Werror (-O0 and -O2 on selected examples, fresh clone)

Byte-for-byte

Runtime spec cases and every command in docs/demonstration.md compared exactly: stdout, stderr, exit status

35,000 floats

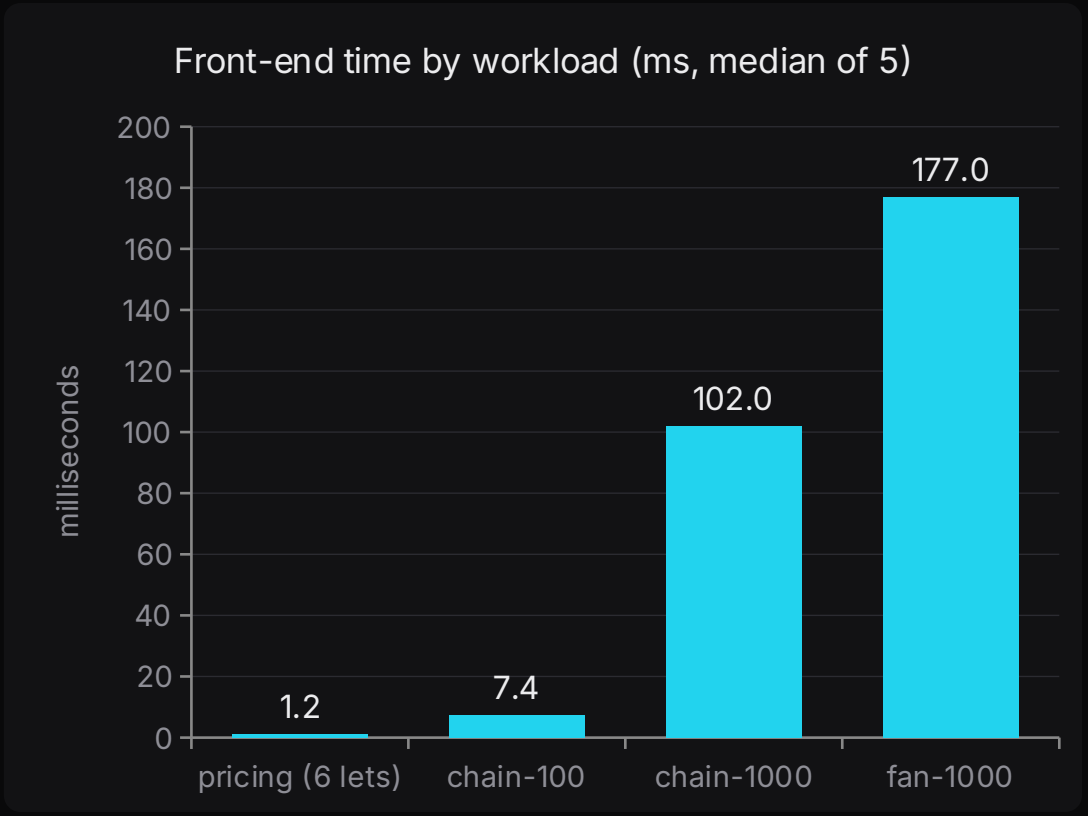
Float formatting checked against Python's repr on every test run

≈12,000 strings

C literal parser for --set compared with the Python lexer

Validated environment: Windows 11, MSYS2 UCRT64 GCC 16.1. Linux/WSL: compiler-side tests passed; generated-C execution was not validated there. No cross-platform claim.

Performance and evaluation



workload	front end	gcc -O0	gcc -O2	run
pricing	1.2	366	451	23.9
chain-100	7.4	438	846	24.8
chain-1000	102	1,651	5,769	27.8
fan-1000	177	2,667	10,561	28.8

milliseconds, median of 5 runs. Three different quantities: CAUSA front end, native C compile, program run.

Compilation dominates

gcc at -O2 took 5.8–10.6 s for 1,000 lets; the CAUSA front end took 0.1–0.2 s.

Runs ≈ process start-up

Executables finish in about 24–29 ms in every workload.

Recorded on one Windows 11 machine (8 logical CPUs, Python 3.11.9, GCC 16.1.0); environment-dependent engineering measurements, not a speed claim.

Limitations and future work

Completed

- Language v1: inputs, lets, show, explain, whatif, impact, assert
- Causal IR with verified, dependency-preserving optimizer
- C99 backend and C runtime
- Launch-time --set inputs
- CAUSA Studio desktop app

Limitations today

- Generated C validated on Windows + MSYS2 UCRT64 GCC only
- No conditionals, loops, functions, strings, collections
- Front end grows faster than linear on long chains
- C compiler dominates build time
- Studio: one file at a time, no launch values

Not built yet

- Test Linux, macOS, Clang, MSVC
- One command to build and run
- Deferred language features
- Studio usability and visualization
- Broader performance evaluation

Sample code: the optimizer's safety check

causa/optimizer.py (excerpt of optimize; comments shortened)

```
    # Remove consts that lost all users.
    used = {o for n in nodes.values() for o in n.operands}
    used |= {s.root for s in steps if isinstance(s, Assert)}
    for node_id in sorted(nodes):
        if nodes[node_id].kind is NodeKind.CONST and node_id not in used:
            del nodes[node_id]
            log.append(Rewrite("remove", node_id, "const node", "no remaining users"))

    result = replace(module, nodes=tuple(nodes.values()), steps=tuple(steps))
    validate_ir(result)  # re-derives the binding-level graph, compares with provenance
    _check_preserved(module, result)
    return OptimizationResult(result, tuple(log))
```

Only constants are removed

A constant cannot raise a runtime error, so deleting unused ones is safe.

Re-derive and compare

validate_ir rebuilds the dependency graph and checks it against provenance.

Fail loudly

Any change to a binding, anchor or action raises an internal compiler error.

Full source: github.com/ujvall/causa

CAUSA connects compilation with explainability and counterfactual reasoning.

`explain`

Understand

why a value was produced

`impact`

Trace

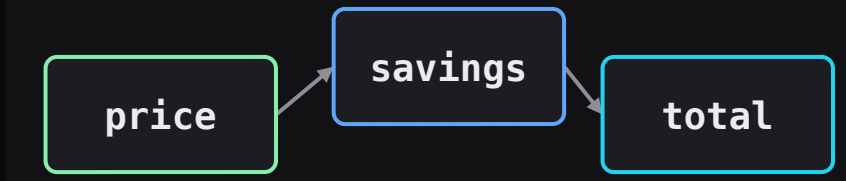
the impact of a binding

`whatif`

Explore

alternative inputs without changing the original execution

Questions?



Source code

github.com/ujvall/causa

`causa/`	compiler pipeline and C runtime
`studio/`	CAUSA Studio desktop application
`examples/`	pricing.cau and other sample programs
`tests/`	automated test suite
`benchmarks/`	benchmark script behind the performance slide
`docs/`	language specification, architecture, demonstration, evaluation

Scan to open the repository

Appendix: references

[1] M. Weiser, “Program Slicing,” IEEE Transactions on Software Engineering, vol. 10, no. 4, pp. 352–357, 1984.

[2] J. Ferrante, K. J. Ottenstein, J. D. Warren, “The Program Dependence Graph and Its Use in Optimization,” ACM TOPLAS, vol. 9, no. 3, pp. 319–349, 1987. doi:10.1145/24039.24041

[3] J. Cheney, L. Chiticariu, W.-C. Tan, “Provenance in Databases: Why, How, and Where,” Foundations and Trends in Databases, vol. 1, no. 4, pp. 379–474, 2009. doi:10.1561/1900000006

[4] J. Cheney, A. Ahmed, U. A. Acar, “Provenance as dependency analysis,” Mathematical Structures in Computer Science, vol. 21, no. 6, pp. 1301–1337, 2011. doi:10.1017/S0960129511000211

[5] A. J. Ko, B. A. Myers, “Designing the Whyline: A Debugging Interface for Asking Questions About Program Behavior,” CHI 2004, Vienna, pp. 151–158.

[6] A.-H. Karimi, G. Barthe, B. Schölkopf, I. Valera, “A Survey of Algorithmic Recourse: Contrastive Explanations and Consequential Recommendations,” ACM Computing Surveys, vol. 55, no. 5, 2022. doi:10.1145/3527848

[7] C. Lattner, V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” CGO 2004, Palo Alto. https://llvm.org/pubs/2004-01-30-CGO-LLVM.html

[8] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, O. Zinenko, “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation,” CGO 2021. https://research.google/pubs/mlir-scaling-compiler-infrastructure-for-domain-specific-computation/

[9] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao, L. Zhang, “A Survey of Compiler Testing,” ACM Computing Surveys, vol. 53, no. 1, pp. 1–36, 2020. doi:10.1145/3363562

[10] H. Ma, “A Survey of Modern Compiler Fuzzing,” arXiv preprint, 2023 (not peer reviewed). doi:10.48550/arXiv.2306.06884